Guía de Implementación — Bastón Inteligente IoT (SafeCane)

CS5055: Internet of Things · UTEC · Guía técnica de construcción paso a paso

Octubre 2026

Índice

0. Cómo usar esta guía **3**
Cambio respecto del informe original . . . 3
Arquitectura resumida . . . 3

1. Requisitos previos **4**
1.1 Software a instalar . . . 4
1.2 Hardware . . . 5
1.3 Herramientas . . . 7
1.4 Antes de comprar: tres verificaciones . . . 7

2. Fase 0 — Preparar el entorno **8**
2.1 Instalar el core ESP32 . . . 8
2.2 Instalar el driver USB . . . 8
2.3 Primera prueba: blink . . . 8

3. Fase 1 — Validar cada sensor por separado **9**
3.1 Mapa de pines definitivo . . . 9
3.2 Paso 1 — HC-SR04 (distancia) . . . 9
3.3 Paso 2 — MPU6050 (caídas) . . . 10
3.4 Paso 3 — FSR 402 en la empuñadura (agarre) . . . 11
3.5 Paso 4 — Piezo en la punta (golpeteo) . . . 12
3.6 Paso 5 — GPS NEO-6M (ubicación) . . . 13
3.7 Paso 6 — LDR, pulsador y actuadores . . . 14

4. Fase 2 — Integración eléctrica **16**
4.1 Orden de integración . . . 16
4.2 Reglas de cableado que evitan fallos . . . 16
4.3 Criterio de aceptación de la Fase 2 . . . 16

5. Fase 3 — Firmware integrado **17**
5.1 Arquitectura temporal . . . 17
5.2 Máquina de estados . . . 17
5.3 Esqueleto del programa . . . 17
5.4 Detección de caídas . . . 18

5.5 Aviso de obstáculos 19
5.6 Criterio de aceptación de la Fase 3 19

6. Fase 4 — Calibración del algoritmo 20
6.1 Registrar datos 20
6.2 Protocolo de captura 20
6.3 Métricas a calcular 20

7. Fase 5 — Comunicación y tablero 22
7.1 Tópicos MQTT 22
7.2 BLE 22
7.3 Tablero en Node-RED 22
7.4 Criterio de aceptación de la Fase 5 23

8. Fase 6 — Alimentación autónoma 24
8.1 Ajustar el MT3608 ANTES de conectar nada 24
8.2 Cadena de alimentación 24
8.3 Medir la autonomía real 24
8.4 Modo de bajo consumo 24

9. Fase 7 — Montaje en el bastón 25
9.1 Distribución física 25
9.2 Orden de armado 25
9.3 Criterio de aceptación de la Fase 7 25

10. Fase 8 — Validación 26
10.1 Tabla de resultados a llenar 26
10.2 Limitaciones a reportar con honestidad 26

Anexo A — Diagnóstico de fallos 27

Anexo B — Cronograma sugerido 28

Anexo C — Lista de verificación final 29

0. Cómo usar esta guía

Esta guía cubre la construcción completa del prototipo descrito en el informe del proyecto: un bastón instrumentado con ESP32 que detecta obstáculos frontales, identifica caídas, registra el uso activo y georreferencia las alertas.

La guía está organizada en **ocho fases secuenciales**. La regla más importante es esta: **no pases a la fase siguiente hasta que la actual cumpla su criterio de aceptación.** Cada fase incluye uno. La causa número uno de proyectos de este tipo que no funcionan es haber soldado todo junto antes de validar cada sensor por separado: cuando algo falla, hay veinte causas posibles y ninguna forma de aislarlas.

Cambio respecto del informe original

Esta guía incorpora una modificación de diseño respecto de la propuesta escrita:

En el informe	En esta guía	Motivo
FSR 402 en la punta del bastón	FSR 402 en la **empuñadura** (fuerza de agarre)	La punta somete al FSR a esfuerzo cortante, impacto repetido (30–60 golpes/min) y humedad. El sensor se destruye en días. Trujillo-León et al. [16] lo colocan en el mango por esta razón.
—	**Disco piezoeléctrico** añadido en la punta	Detecta el golpeteo de la contera contra el suelo. El impacto *es* la señal, así que el impacto no lo daña.

El beneficio no es solo mecánico. Con el FSR en la punta, un bastón que se cae solo y un usuario que se cae producen la misma señal (deja de haber apoyo). Con el sensor de agarre en la empuñadura, el algoritmo distingue ambos casos: si hay impacto en la IMU **y** la mano sigue sujetando, se cayó la persona; si hay impacto y no hay agarre, se cayó el bastón. Esto cumple de forma verificable el objetivo específico 3 del informe.

Arquitectura resumida

Capa	Elementos
Percepción	HC-SR04, MPU6050, FSR 402, piezo, NEO-6M, LDR, pulsador
Procesamiento	ESP32-WROOM-32 (filtrado, umbrales, máquina de estados)
Actuación	Motor vibrador (PWM), buzzer, LED
Red	BLE (app móvil) y Wi-Fi/MQTT (tablero)
Aplicación	App móvil + Node-RED o Grafana

1. Requisitos previos

1.1 Software a instalar

Instala todo esto **antes** de tocar el hardware. Son descargas gratuitas.

Programa	Versión	Para qué	Dónde
Arduino IDE	2.3.x o superior	Entorno de programación principal	arduino.cc/en/software
Core ESP32 de Espressif	3.x	Soporte del chip dentro del IDE	Se instala desde el propio IDE (ver §2.1)
Driver USB-serie	CP210x o CH340	Que la PC reconozca la placa	Según el chip de tu DevKit (ver §2.2)
Node-RED	4.x	Tablero de monitoreo	nodered.org (requiere Node.js 18+)
Node.js	18 LTS o superior	Base de Node-RED	nodejs.org
Mosquitto	2.x	Broker MQTT local	mosquitto.org
MQTT Explorer	Cualquiera	Depurar los tópicos MQTT. Imprescindible.	mqtt-explorer.com
nRF Connect for Mobile	App móvil	Depurar BLE antes de tener la app propia	Play Store / App Store

Alternativa a instalar Mosquitto localmente: usa un broker público de pruebas (`broker.emqx.io`, puerto 1883) o una instancia gratuita de HiveMQ Cloud. Para la demostración en clase el broker público es más rápido de montar, pero no tiene autenticación: cualquiera puede ver tus tópicos. Para la entrega final usa Mosquitto local con usuario y contraseña, que es la medida mínima de seguridad que el informe compromete en sus alcances.

Librerías de Arduino

Se instalan desde **Herramientas → Administrar bibliotecas** dentro del IDE:

Librería	Autor	Uso
TinyGPSPlus	Mikal Hart	Decodificar sentencias NMEA del GPS
PubSubClient	Nick O'Leary	Cliente MQTT
NimBLE-Arduino	h2zero	BLE (mucho más liviano que el BLE estándar del core)
ArduinoJson	Benoit Blanchon	Armar los mensajes JSON de las alertas

El MPU6050 se lee con registros I2C directos usando la librería `Wire` incluida en el core, sin dependencias externas. El HC-SR04 se lee con `pulseIn()`, también nativo.

1.2 Hardware

Procesamiento

Componente	Especificación	Cant.
ESP32-WROOM-32 DevKit V1, **38 pines**	Doble núcleo 240 MHz, Wi-Fi + BLE, ADC 12 bits	1

La versión de 30 pines no expone varios ADC1 que vas a necesitar. Evita la variante WROOM-32**U**, que requiere antena externa.

Sensores

Componente	Especificación	Cant.
HC-SR04	5 V, 40 kHz, 2–400 cm, cono ≈15°	1
MPU6050 (módulo GY-521)	Acelerómetro ±2/4/8/16 g, giroscopio, I2C	1
FSR 402 (Interlink)	Área activa 12,7 mm, 0,1–10 N	1
Disco piezoeléctrico	27 mm, con cables soldados	1
Módulo GPS NEO-6M (GY-NEO6MV2)	2,5 m CEP, UART 9600 bps, antena cerámica	1
LDR GL5528	8–20 kΩ a 10 lux	1
Pulsador táctil + capuchón alto	12 mm, normalmente abierto	1

Actuadores

Componente	Especificación	Cant.
Motor vibrador ERM tipo moneda	3 V, 60–90 mA, 10×3 mm	1
Buzzer **activo**	5 V, ≈85 dB a 10 cm	1
LED blanco de alto brillo	5 mm, 20 mA	1

Alimentación

Componente	Especificación	Cant.
Celda Li-ion 18650	3,7 V, 3000 mAh **reales**	1
Portaceldas 18650	Con cables	1
Módulo TP4056 **con protección**	Debe traer chips DW01A + FS8205A	1
Convertidor elevador MT3608	Entrada 2–24 V, salida ajustable, 2 A	1
Interruptor deslizante SPDT	3 A	1

Componente	Especificación	Cant.
Cable micro-USB	Datos, no solo carga	1

Desconfía de las celdas 18650 que anuncian 6000–9800 mAh: son falsas, y la autonomía calculada en el informe asume 3000 mAh reales. Marcas verificables: Samsung 30Q, LG MJ1, Sony VTC.

Componentes pasivos y discretos

Componente	Valor	Cant.	Dónde va
Resistencia	1 kΩ	3	Divisor del Echo + base de los 2 transistores
Resistencia	2 kΩ (o 2×1 kΩ en serie)	1	Divisor del Echo, parte baja
Resistencia	10 kΩ	4	Divisores de FSR y LDR, pull-up del pulsador, reserva
Resistencia	1 MΩ	1	Descarga del piezo
Resistencia	100 kΩ	1	Serie de protección del piezo
Resistencia	100–220 Ω	1	Limitadora del LED
Transistor NPN 2N2222	IC máx. 600 mA	2	Driver de motor y de buzzer
Diodo 1N4007	Rectificador	2	Rueda libre en motor y buzzer
Diodo 1N4148	Señal, rápido	2	Recorte del piezo a 3,3 V y a GND
Capacitor electrolítico	100–470 µF / 10 V	2	Rieles de 5 V y 3,3 V
Capacitor cerámico	100 nF	4	Desacoplo junto a cada módulo

Los dos electrolíticos no son opcionales: son lo que evita que el módulo GPS se reinicie cada vez que arranca el motor vibrador.

Mecánica y cableado

Componente	Especificación	Cant.
Bastón plegable de aluminio	Con contera desmontable	1
Protoboard	830 puntos	1
Perfboard / PCB perforada	7×9 cm	2
Jumpers Dupont	M-M, M-H, H-H (40 c/u)	3 juegos
Espadines header	Macho y hembra, tira de 40	2

Componente	Especificación	Cant.
Cable flexible AWG 26–28	4 conductores, ≈1,2 m	1
Cable apantallado o par trenzado	2 conductores, ≈1,2 m (al piezo)	1
Carcasa impresa en 3D (PLA)	Empuñadura + alojamiento de contera	2 piezas
Termorretráctil	Surtido 2–6 mm	1 juego
Bridas y velcro adhesivo	—	1 juego
Tornillos M3 y espaciadores	8–12 mm	1 juego
Almohadilla de silicona o EVA	3 mm de espesor	1

1.3 Herramientas

Cautín de 30–40 W o estación, estaño con flux, flux en pasta, bomba desoldadora, pelacables, pinza de punta fina, **multímetro** (indispensable: sin él no puedes ajustar el MT3608 ni medir el consumo), pistola de silicona caliente, alcohol isopropílico, cinta kapton o aislante.

1.4 Antes de comprar: tres verificaciones

1. **El TP4056 debe tener protección.** Las placas sin protección no tienen los chips DW01A y FS8205A y dejan que la celda se descargue por debajo de 2,5 V, lo que la arruina de forma permanente. Pídelo explícitamente “con protección”.
2. **El módulo NEO-6M debe traer antena cerámica** y, de preferencia, la batería de respaldo ML1220 viva. Si está muerta, cada encendido es un arranque en frío de unos 27 segundos en lugar de 1–2 s.
3. **El FSR 402 genuino de Interlink** no siempre está disponible localmente; lo que circula suele ser un clon con tolerancias peores. Para un umbral binario el clon sirve, pero no esperes repetibilidad entre unidades.

2. Fase 0 — Preparar el entorno

Objetivo: que la PC programe el ESP32 y que el chip ejecute código.

2.1 Instalar el core ESP32

1. Abre Arduino IDE → **Archivo → Preferencias**.
2. En "URLs adicionales de gestor de tarjetas" pega: `https://espressif.github.io/arduino-esp32/package_esp32_index.json`
3. **Herramientas → Placa → Gestor de tarjetas**, busca `esp32` e instala el paquete de **Espressif Systems** (no el de terceros). Pesa cerca de 1 GB.
4. Selecciona **Herramientas → Placa → ESP32 Arduino → ESP32 Dev Module**.

2.2 Instalar el driver USB

Mira el chip cuadrado pequeño junto al conector USB de tu placa:

- Dice **CP2102** → instala el driver de Silicon Labs.
- Dice **CH340** o **CH9102** → instala el driver de WCH.

Conecta la placa y verifica que aparece un puerto nuevo en **Herramientas → Puerto**. Si no aparece ningún puerto: casi siempre es un **cable USB de solo carga**. Prueba con otro cable antes de buscar cualquier otra causa.

2.3 Primera prueba: blink

```
void setup() {
  pinMode(2, OUTPUT);          // LED azul integrado del DevKit
  Serial.begin(115200);
}

void loop() {
  digitalWrite(2, HIGH); delay(500);
  digitalWrite(2, LOW);  delay(500);
  Serial.println("ESP32 vivo");
}
```

Sube el código con el botón de flecha. Abre el Monitor Serie a **115200 baudios**.

> **Si la carga se queda en `Connecting.....____`:** mantén presionado el botón **BOOT** de la placa mientras aparece ese mensaje y suéltalo cuando empiece a subir. Algunos DevKit necesitan esto siempre.

Criterio de aceptación de la Fase 0: el LED parpadea y el Monitor Serie imprime el mensaje de forma legible (si salen caracteres basura, revisa la velocidad).

3. Fase 1 — Validar cada sensor por separado

Todo en protoboard. Un sensor a la vez. **No conectes dos sensores hasta que cada uno funcione solo.** Cada paso tiene un criterio de aceptación concreto.

3.1 Mapa de pines definitivo

Úsalo desde ahora para no recablear después. Está elegido para evitar los tres problemas clásicos del ESP32.

Señal	GPIO	Modo	Por qué ese pin
HC-SR04 Trig	5	Salida	Libre, sin función de arranque
HC-SR04 Echo	18	Entrada, tras divisor	—
MPU6050 SDA	21	I2C	Pin I2C por defecto
MPU6050 SCL	22	I2C	Pin I2C por defecto
GPS → ESP32 (RX2)	16	UART2	Deja UART0 libre para programar
ESP32 → GPS (TX2)	17	UART2	—
FSR empuñadura	34	ADC1_CH6	**ADC1**: funciona con Wi-Fi activo
Piezo punta	35	ADC1_CH7	ADC1
LDR	32	ADC1_CH4	ADC1
Pulsador	27	Entrada, `INPUT_PULLUP`	Tiene pull-up interno
Motor vibrador	25	PWM (LEDC)	—
Buzzer	26	Salida	—
LED	33	Salida	—

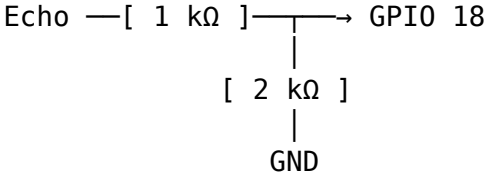
Tres reglas que explican este mapa:

- **ADC2 no funciona con Wi-Fi activo.** Todas las entradas analógicas deben ir en ADC1 (GPIO 32–39). Si pones el FSR en un pin ADC2, las lecturas se congelan en cuanto conectes el MQTT, y es un fallo dificilísimo de diagnosticar.
- **GPIO 34–39 son solo entrada** y no tienen pull-up interno. No sirven para salidas ni para el pulsador.
- **GPIO 6–11** están ocupados por la memoria flash. **GPIO 0, 2, 12 y 15** son pines de *strapping*: si tienen algo conectado al arrancar, la placa puede no bootear.

Quedan libres GPIO 4, 13, 14, 19 y 23 por si necesitas ampliar.

3.2 Paso 1 — HC-SR04 (distancia)

Conexión. VCC al riel de 5 V, GND a GND, Trig a GPIO 5. El pin Echo entrega 5 V y el ESP32 tolera 3,3 V: va **obligatoriamente** por un divisor.



Esto da 5 V × 2/(1+2) = 3,33 V. Conectar Echo directo al GPIO daña el pin de forma progresiva: funciona unos días y luego falla.

```
const int TRIG = 5, ECHO = 18;

void setup() {
  Serial.begin(115200);
  pinMode(TRIG, OUTPUT);
  pinMode(ECHO, INPUT);
}

float medirDistancia() {
  digitalWrite(TRIG, LOW);  delayMicroseconds(2);
  digitalWrite(TRIG, HIGH); delayMicroseconds(10);
  digitalWrite(TRIG, LOW);
  long t = pulseIn(ECHO, HIGH, 25000);   // timeout 25 ms = ~4 m
  if (t == 0) return -1.0;               // sin eco
  return (t * 0.0343) / 2.0;             // cm
}

void loop() {
  float d = medirDistancia();
  Serial.println(d);
  delay(200);
}
```

Criterio de aceptación. Mide con una cinta métrica a 20, 50, 100 y 200 cm contra una pared lisa. El error debe ser menor a 3 cm y las lecturas estables (variación menor a 2 cm entre muestras consecutivas).

Nota experimental para el informe. Repite la medición contra tres materiales: pared de concreto, vidrio y tela o ropa. Vas a ver que la tela absorbe el ultrasonido y devuelve lecturas erráticas o ningún eco. Eso confirma lo que reporta Abuelmakarem et al. [11] y es un dato que vale documentar en la sección de resultados.

3.3 Paso 2 — MPU6050 (caídas)

Conexión. VCC a 3,3 V, GND a GND, SDA a GPIO 21, SCL a GPIO 22. El módulo GY-521 ya trae regulador y resistencias pull-up de I2C; no agregues más.

Primero verifica que el chip responde con un escáner I2C:

```
#include <Wire.h>

void setup() {
  Serial.begin(115200);
  Wire.begin(21, 22);
  for (byte a = 1; a < 127; a++) {
    Wire.beginTransmission(a);
    if (Wire.endTransmission() == 0) {
      Serial.print("Dispositivo en 0x");
      Serial.println(a, HEX);
    }
  }
}
void loop() {}
```

Debe aparecer **0x68** (o 0x69 si el pin AD0 está a VCC). Si no aparece nada, están cruzados SDA y SCL, o el módulo no tiene alimentación.

Ahora la lectura. **Punto crítico:** hay que configurar el acelerómetro en **±8 g**. Por

defecto arranca en ±2 g, y un impacto de caída supera los 2,5 g, así que se saturaría justo en el evento que quieres detectar. Esto no está en el informe y es el error más frecuente al implementar detección de caídas.

```
#include <Wire.h>
const int MPU = 0x68;
const float LSB_8G = 4096.0;   // sensibilidad en ±8 g

void initMPU() {
  Wire.begin(21, 22);
  Wire.beginTransmission(MPU);
  Wire.write(0x6B); Wire.write(0x00);   // despertar
  Wire.endTransmission(true);
  Wire.beginTransmission(MPU);
  Wire.write(0x1C); Wire.write(0x10);   // ACCEL_CONFIG = ±8 g
  Wire.endTransmission(true);
  Wire.beginTransmission(MPU);
  Wire.write(0x1A); Wire.write(0x03);   // DLPF 44 Hz, filtra vibracion
  Wire.endTransmission(true);
}

void leerAcel(float &ax, float &ay, float &az) {
  Wire.beginTransmission(MPU);
  Wire.write(0x3B);
  Wire.endTransmission(false);
  Wire.requestFrom(MPU, 6, true);
  int16_t rx = Wire.read() << 8 | Wire.read();
  int16_t ry = Wire.read() << 8 | Wire.read();
  int16_t rz = Wire.read() << 8 | Wire.read();
  ax = rx / LSB_8G;  ay = ry / LSB_8G;  az = rz / LSB_8G;
}

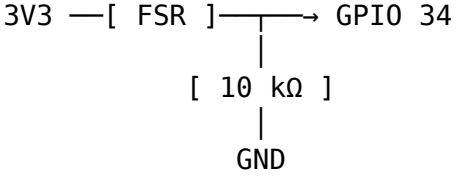
void setup() { Serial.begin(115200); initMPU(); }

void loop() {
  float ax, ay, az;
  leerAcel(ax, ay, az);
  float svm = sqrt(ax*ax + ay*ay + az*az);
  float incl = acos(constrain(az / svm, -1.0, 1.0)) * 180.0 / PI;
  Serial.print(svm); Serial.print("  "); Serial.println(incl);
  delay(20);                        // 50 Hz
}
```

Criterio de aceptación. Con el módulo quieto sobre la mesa, la SVM debe leer entre 0,97 y 1,03 g. Al sacudirlo bruscamente debe superar 3 g. Al voltearlo 90°, la inclinación debe acercarse a 90°.

3.4 Paso 3 — FSR 402 en la empuñadura (agarre)

Conexión. Divisor de tensión: un terminal del FSR a 3,3 V, el otro a GPIO 34 y también a GND a través de una resistencia de 10 kΩ.



```
const int FSR = 34;

void setup() {
  Serial.begin(115200);
  analogReadResolution(12);              // 0..4095
  analogSetPinAttenuation(FSR, ADC_11db); // rango hasta ≈3,1 V
}

void loop() {
  int v = analogRead(FSR);
  Serial.println(v);
  delay(100);
}
```

Calibración. Anota el valor en tres condiciones: sin tocar (debe rondar 0–100), agarre suave y agarre firme. El umbral de "mano presente" se fija aproximadamente en el 30 % del valor de agarre suave. Repite esto con cada persona que vaya a probar el bastón: la variabilidad entre individuos es alta, y es exactamente lo que reporta Trujillo-León et al. [16]; por eso la literatura recomienda umbrales personalizados.

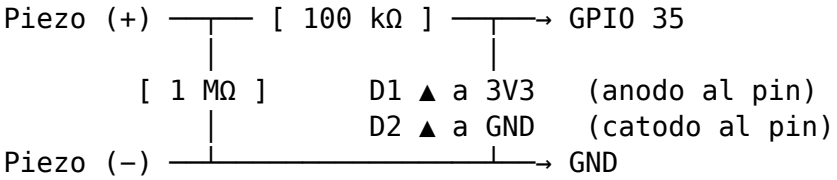
Montaje. Pega el FSR a la empuñadura con una almohadilla de silicona o EVA de 3 mm encima, para repartir la presión de los dedos sobre el área activa. Sin la almohadilla, la lectura cambia drásticamente según dónde caiga cada dedo.

Criterio de aceptación. La lectura distingue de forma repetible entre mano puesta y mano retirada, con al menos 300 cuentas de separación entre ambos estados.

3.5 Paso 4 — Piezo en la punta (golpeteo)

Este sensor no está en el informe original; se añade por el cambio de diseño de §0.

Circuito de acondicionamiento. El piezo genera picos de decenas de voltios. Sin protección destruyes el pin del ESP32.



- La resistencia de **1 MΩ** en paralelo descarga la carga acumulada y define la constante de tiempo. **Suéldala en el piezo mismo, en la punta del bastón**, no en la placa: eso es lo que evita que el metro de cable capte ruido.
- La de **100 kΩ** en serie limita la corriente hacia los diodos.
- Los dos **1N4148** recortan cualquier excursión fuera del rango 0–3,3 V.

Montaje mecánico. Pega el disco con epóxico a la cara interna del tubo, justo encima de la contera. Debe quedar rígidamente acoplado: si queda suelto, no transmite el impacto.

```
const int PIEZO = 35;
const int UMBRAL = 400;         // ajustar en calibracion
unsigned long ultimoGolpe = 0;

void setup() {
```

```
  Serial.begin(115200);
  analogReadResolution(12);
}

void loop() {
  int v = analogRead(PIEZO);
  if (v > UMBRAL && millis() - ultimoGolpe > 150) {  // antirrebote
    ultimoGolpe = millis();
    Serial.print("Golpe, amplitud = ");
    Serial.println(v);
  }
}
```

Calibración. Golpea la punta sobre losa, sobre tierra y sobre pasto. Las amplitudes son muy distintas; fija el umbral por debajo del más débil de los tres (pasto) pero por encima del ruido en reposo. Los 150 ms de antirrebote evitan contar varias veces el mismo golpe por el repique del disco.

Criterio de aceptación. Detecta cada golpe de la punta contra el suelo sin contarlo doble, y no dispara con el bastón quieto en la mano.

3.6 Paso 5 — GPS NEO-6M (ubicación)

Conexión. VCC a 5 V (el módulo tiene regulador propio), GND a GND, **TX del módulo a GPIO 16**, **RX del módulo a GPIO 17**. Fíjate en el cruce: TX del GPS va al RX del ESP32.

Primero una prueba cruda, para ver si llegan datos:

```
void setup() {
  Serial.begin(115200);
  Serial2.begin(9600, SERIAL_8N1, 16, 17);
}
void loop() {
  while (Serial2.available()) Serial.write(Serial2.read());
}
```

Deben aparecer líneas NMEA como `$GPGGA,...` y `$GPRMC,....` Si no aparece nada, están cruzados TX y RX. Si aparecen líneas pero con campos vacíos, el módulo funciona pero todavía no fija satélites.

Luego, decodificado:

```
#include <TinyGPSPlus.h>
TinyGPSPlus gps;

void setup() {
  Serial.begin(115200);
  Serial2.begin(9600, SERIAL_8N1, 16, 17);
}

void loop() {
  while (Serial2.available()) gps.encode(Serial2.read());
  if (gps.location.isUpdated()) {
    Serial.print(gps.location.lat(), 6); Serial.print(", ");
    Serial.print(gps.location.lng(), 6); Serial.print("  sats=");
    Serial.println(gps.satellites.value());
```

```
  }
}
```

> **Lo más importante del GPS: tiene que estar al aire libre.** Dentro de un laboratorio no va a fijar nunca. Sácalo a un patio, con el lado de la antena cerámica hacia arriba y sin obstrucciones. El primer arranque en frío tarda entre 30 segundos y 2 minutos. El LED del módulo parpadea una vez por segundo cuando ya tiene fix; mientras está apagado o fijo, todavía no.

Criterio de aceptación. Obtiene fix al aire libre en menos de 2 minutos, reporta al menos 4 satélites, y las coordenadas corresponden a tu ubicación real al pegarlas en Google Maps. Compara el error con los 4,63 m de RMSE radial que reportan Fitriandini et al. [10]: es tu referencia de comparación.

3.7 Paso 6 — LDR, pulsador y actuadores

LDR. Igual que el FSR: divisor con 10 kΩ a GND, salida a GPIO 32. Verifica que el valor cambia al tapar el sensor con la mano.

Pulsador. Un terminal a GPIO 27, el otro a GND. Usa `pinMode(27, INPUT_PULLUP)` y antirrebote por software de 50 ms. Ponle un capuchón alto o en relieve: el usuario es ciego y tiene que encontrarlo al tacto, bajo presión, después de una caída.

Motor vibrador. No lo conectes directo a un GPIO: consume 60–90 mA y el pin entrega cómodamente 12 mA.

```
GPIO 25 ──[ 1 kΩ ]── Base (2N2222)
                     Colector ── Motor ── 3V3
                     Emisor ──── GND
Diodo 1N4007 en paralelo al motor, catodo hacia 3V3.
```

El diodo de rueda libre es obligatorio: sin él, el pico inductivo al apagar el motor reinicia el ESP32 o corrompe las lecturas del GPS.

```
const int MOTOR = 25;

void setup() {
  ledcAttach(MOTOR, 5000, 8);   // core 3.x: pin, 5 kHz, 8 bits
}

void loop() {
  for (int d = 0; d <= 255; d += 5) { ledcWrite(MOTOR, d); delay(50); }
  ledcWrite(MOTOR, 0); delay(1000);
}
```

> Si usas el **core 2.x** el API es distinto: `ledcSetup(0, 5000, 8); ledcAttachPin(MOTOR, 0); ledcWrite(0, duty);` (el primer argumento es el canal, no el pin). Si el compilador se queja de `ledcAttach`, ese es el motivo.

Buzzer. Mismo esquema con el segundo 2N2222, pero alimentado del riel de 5 V. Consume unos 30 mA, por encima del límite cómodo del GPIO.

LED. Con Vf de 3,0–3,2 V casi no hay margen desde un pin de 3,3 V. Aliméntalo del riel de 5 V con la resistencia de 100–220 Ω, conmutado por el mismo tipo de driver o compartiendo el del buzzer si no necesitas usarlos de forma independiente.

Criterio de aceptación de la Fase 1. Los siete sensores y los tres actuadores funcionan **individualmente**, cada uno con su sketch de prueba. Guarda cada sketch en una carpeta separada: los vas a necesitar para diagnosticar fallos después.

4. Fase 2 — Integración eléctrica

Objetivo: todo conectado a la vez, sin alimentación de batería todavía (sigue alimentando por USB).

4.1 Orden de integración

Conecta de a un sensor, verificando después de cada uno que los anteriores siguen funcionando. Este orden pone primero lo que más molesta:

1. MPU6050 (I2C)
2. HC-SR04 (consume picos de corriente)
3. GPS (sensible a ruido de alimentación)
4. Entradas analógicas: FSR, piezo, LDR
5. Pulsador
6. Actuadores: motor, buzzer, LED

4.2 Reglas de cableado que evitan fallos

- **Un solo punto de GND común.** Todos los GND —ESP32, sensores, batería, elevador— se unen en un mismo nodo. Masas separadas producen lecturas analógicas sin sentido.
- **Capacitor de 100 nF junto a cada módulo**, entre su VCC y su GND, lo más cerca posible de los pines del módulo.
- **Electrolítico de 100–470 µF en cada riel** (5 V y 3,3 V).
- **Cablea el piezo con par trenzado o apantallado**, separado del cable del motor. El motor es la principal fuente de ruido del sistema.
- **Prueba el motor mientras el GPS está fijando.** Si el GPS pierde el fix al vibrar, te falta capacidad de desacoplo.

4.3 Criterio de aceptación de la Fase 2

Un sketch que lee los siete sensores y los imprime al Monitor Serie, con el motor vibrando de forma intermitente, sin que ninguna lectura se degrade ni el ESP32 se reinicie, durante **10 minutos continuos**.

5. Fase 3 — Firmware integrado

Objetivo: un solo programa, no bloqueante, con máquina de estados.

5.1 Arquitectura temporal

Nada de `delay()` en el programa principal: usa `millis()`. Cada tarea corre a su propia frecuencia.

Tarea	Periodo	Motivo
Leer IMU	20 ms (50 Hz)	Mínimo para capturar el pico de impacto de una caída
Leer piezo	5 ms	Los picos del impacto son muy breves
Medir distancia	100 ms	Suficiente a velocidad de marcha; `pulseIn` bloquea hasta 25 ms
Leer FSR y LDR	200 ms	Cambian lento
Procesar GPS	Continuo	Vaciar el buffer de UART2 cada vuelta
Actualizar vibración	50 ms	Percepción háptica
Publicar estado por MQTT	5 s	No saturar la red ni la batería

5.2 Máquina de estados

```
NORMAL ──(posible caida detectada)──→ PRE_ALERTA
PRE_ALERTA ──(pulsador en 10 s)──→ NORMAL
PRE_ALERTA ──(timeout 10 s)──→ ALERTA
ALERTA ──(pulsador)──→ NORMAL
NORMAL ──(sin golpes de piezo por 5 min)──→ REPOSO
REPOSO ──(golpe de piezo o movimiento)──→ NORMAL
```

El intervalo de cancelación de 10 s es el mecanismo de confirmación interactiva que describe Geng [14] y es lo que mantiene bajas las falsas alarmas. En `PRE_ALERTA` el bastón vibra con un patrón distintivo para avisar al usuario que está por emitir la alerta.

5.3 Esqueleto del programa

```
// ---------- Pines ----------
const int TRIG=5, ECHO=18, SDA_P=21, SCL_P=22;
const int GPS_RX=16, GPS_TX=17;
const int FSR=34, PIEZO=35, LDR=32, BOTON=27;
const int MOTOR=25, BUZZER=26, LED=33;

// ---------- Umbrales (calibrar en Fase 4) ----------
const float SVM_CAIDA_LIBRE = 0.5;   // g
const float SVM_IMPACTO     = 2.5;   // g
const float INCLIN_MIN      = 60.0;  // grados
const unsigned long VENTANA_IMPACTO = 1000;  // ms
const unsigned long INCLIN_SOSTENIDA = 2000; // ms
```

```
const unsigned long T_CANCELACION   = 10000; // ms
const int UMBRAL_FSR   = 800;        // calibrar por usuario
const int UMBRAL_PIEZO = 400;        // calibrar por superficie

enum Estado { NORMAL, PRE_ALERTA, ALERTA, REPOSO };
Estado estado = NORMAL;

unsigned long tIMU=0, tPiezo=0, tDist=0, tLento=0, tVibra=0, tMqtt=0;
unsigned long tCaidaLibre=0, tInclinacion=0, tPreAlerta=0, tUltimoGolpe=0;
bool huboCaidaLibre=false;
float distancia=999, svm=1.0, inclinacion=0;
bool manoPresente=false, usoActivo=false;

void setup() {
  Serial.begin(115200);
  Serial2.begin(9600, SERIAL_8N1, GPS_RX, GPS_TX);
  pinMode(TRIG, OUTPUT); pinMode(ECHO, INPUT);
  pinMode(BOTON, INPUT_PULLUP);
  pinMode(BUZZER, OUTPUT); pinMode(LED, OUTPUT);
  ledcAttach(MOTOR, 5000, 8);
  analogReadResolution(12);
  initMPU();
  conectarWiFi();
  initBLE();
}

void loop() {
  unsigned long ahora = millis();

  procesarGPS();                      // vaciar UART cada vuelta

  if (ahora - tPiezo >= 5)   { tPiezo = ahora; tareaPiezo(); }
  if (ahora - tIMU   >= 20)  { tIMU   = ahora; tareaIMU(); }
  if (ahora - tDist  >= 100) { tDist  = ahora; tareaDistancia(); }
  if (ahora - tLento >= 200) { tLento = ahora; tareaLenta(); }
  if (ahora - tVibra >= 50)  { tVibra = ahora; tareaVibracion(); }
  if (ahora - tMqtt  >= 5000){ tMqtt  = ahora; publicarEstado(); }

  maquinaEstados(ahora);
  atenderBoton(ahora);
}
```

5.4 Detección de caídas

```
void tareaIMU() {
  float ax, ay, az;
  leerAcel(ax, ay, az);
  svm = sqrt(ax*ax + ay*ay + az*az);
  inclinacion = acos(constrain(az / max(svm, 0.01f), -1.0f, 1.0f))
                * 180.0 / PI;

  unsigned long ahora = millis();

  // Fase 1: caida libre
  if (svm < SVM_CAIDA_LIBRE) {
    huboCaidaLibre = true;
    tCaidaLibre = ahora;
  }

  // Fase 2: impacto dentro de la ventana
```

```
  if (huboCaidaLibre && (ahora - tCaidaLibre) < VENTANA_IMPACTO
      && svm > SVM_IMPACTO) {
    huboCaidaLibre = false;
    tInclinacion = ahora;                // arranca fase 3
  }

  // Expira la ventana sin impacto
  if (huboCaidaLibre && (ahora - tCaidaLibre) >= VENTANA_IMPACTO) {
    huboCaidaLibre = false;
  }

  // Fase 3: inclinacion sostenida + discriminador de agarre
  if (tInclinacion > 0 && estado == NORMAL) {
    if (inclinacion < INCLIN_MIN) {
      tInclinacion = 0;                  // se reincorporo: falsa alarma
    } else if (ahora - tInclinacion >= INCLIN_SOSTENIDA) {
      tInclinacion = 0;
      if (manoPresente) {                // <<< discriminador clave
        estado = PRE_ALERTA;
        tPreAlerta = ahora;
      }
      // sin mano en la empunadura: se cayo el baston, no la persona
    }
  }
}
```

La condición `manoPresente` es la aportación del cambio de diseño: es lo que separa “se cayó el usuario” de “se cayó el bastón”. Sin ella, el sistema genera una falsa alarma cada vez que el usuario apoya el bastón contra una pared y este resbala.

5.5 Aviso de obstáculos

```
void tareaVibracion() {
  if (estado != NORMAL) return;        // en alerta manda el buzzer
  static bool pulso = false;
  pulso = !pulso;

  if (distancia > 0 && distancia < 50)       ledcWrite(MOTOR, 255);
  else if (distancia < 100) ledcWrite(MOTOR, pulso ? 200 : 0);
  else if (distancia < 200) ledcWrite(MOTOR, pulso ? 120 : 0);
  else                      ledcWrite(MOTOR, 0);
}
```

Las tres zonas (vibración continua intensa, pulsos rápidos, pulsos lentos) corresponden a lo especificado en la sección V-D del informe. Que el canal principal sea háptico y no sonoro es deliberado: Dakopoulos y Bourbakis [3] advierten que el audio continuo enmascara los sonidos del entorno de los que depende una persona ciega para orientarse.

5.6 Criterio de aceptación de la Fase 3

El bastón, alimentado por USB, vibra correctamente según la distancia, transita por los cuatro estados de forma observable en el Monitor Serie, y permite cancelar una pre-alerta con el pulsador. El GPS mantiene el fix mientras el motor vibra.

6. Fase 4 — Calibración del algoritmo

Los umbrales del informe (0,5 g de caída libre, 2,5–3 g de impacto, 60° de inclinación) son **valores de diseño**, no constantes universales. Hay que calibrarlos con datos del propio bastón. Esta fase es la que produce la evidencia experimental de la entrega.

6.1 Registrar datos

Añade al firmware un modo de registro que imprima por serie, a 50 Hz:

`millis, ax, ay, az, svm, inclinacion, fsr, piezo`

Captura el Monitor Serie a un archivo `.csv` y grafícalo en Python o Excel.

6.2 Protocolo de captura

Registra al menos **20 repeticiones de cada escenario**:

Escenario	Debe generar alerta	Repeticiones
Caída hacia adelante (sobre colchoneta)	Sí	20
Caída lateral	Sí	20
Caída hacia atrás	Sí	20
Marcha normal con el bastón	No	5 min
Sentarse bruscamente	No	20
Golpear la punta fuerte contra el suelo	No	20
Subir y bajar escaleras	No	20
Soltar el bastón al suelo (sin caerse)	**No**	20
Apoyar el bastón contra la pared y que resbale	**No**	10

Las dos últimas filas son las que validan el discriminador de agarre. Son la prueba experimental del objetivo específico 3 del informe y lo que distingue tu trabajo de los antecedentes revisados.

> **Seguridad.** Las caídas se simulan sobre colchonetas y nunca con adultos mayores ni personas con discapacidad visual. Para la validación con usuarios reales, limita las pruebas a la marcha y a la detección de obstáculos; las caídas se simulan con maniquí o con integrantes del equipo sobre superficie acolchada.

6.3 Métricas a calcular

Con los datos etiquetados:

- **Sensibilidad** = VP / (VP + FN) — el objetivo específico 2 pide ≥ 90 %.
- **Especificidad** = VN / (VN + FP).
- **Falsas alarmas por hora** durante marcha normal.

Si la sensibilidad queda baja, baja el umbral de impacto. Si hay demasiadas falsas alarmas, sube el umbral o alarga el tiempo de inclinación sostenida. Documenta los valores iniciales, los finales y por qué cambiaron: ese razonamiento es lo que se evalúa en el informe final.

Guarda el `.csv`: según Shri Harini et al. [8], el camino de mejora es reemplazar los umbrales por un clasificador entrenado (un perceptrón multicapa), y para eso vas a necesitar justamente estos datos etiquetados.

7. Fase 5 — Comunicación y tablero

7.1 Tópicos MQTT

Tópico	Contenido	Frecuencia
`baston/estado`	JSON: estado, distancia, batería, uso activo	Cada 5 s
`baston/alerta`	JSON: tipo, timestamp, lat, lon	Solo al confirmar alerta
`baston/gps`	JSON: lat, lon, satélites, HDOP	Cada 30 s

```
#include <WiFi.h>
#include <PubSubClient.h>
#include <ArduinoJson.h>

WiFiClient espClient;
PubSubClient mqtt(espClient);

void publicarAlerta(const char* tipo) {
  StaticJsonDocument<256> doc;
  doc["tipo"]  = tipo;
  doc["ts"]    = millis();
  doc["lat"]   = gps.location.lat();
  doc["lon"]   = gps.location.lng();
  doc["sats"]  = gps.satellites.value();
  char buf[256];
  serializeJson(doc, buf);
  mqtt.publish("baston/alerta", buf, true);   // retained
}
```

Publica la alerta con el flag **retained** en `true`: así, si el tablero se conecta después del evento, igual recibe la última alerta.

Punto importante: `mqtt.loop()` debe llamarse en cada vuelta del `loop()` principal, y la reconexión no puede bloquear. Si pierdes el Wi-Fi, el bastón tiene que seguir vibrando ante obstáculos y sonando ante caídas: la alerta local nunca debe depender de la red.

7.2 BLE

Con NimBLE, un servicio con dos características: una de notificación para alertas y estado, y otra de escritura para configurar umbrales desde la app. Antes de programar la app, verifica con **nRF Connect** desde el celular que el ESP32 anuncia el servicio y emite las notificaciones.

7.3 Tablero en Node-RED

Flujo mínimo:

1. Nodo `mqtt in` suscrito a `baston/#`.
2. Nodo `json` para parsear.

3. `ui_gauge` para la distancia, `ui_text` para el estado, `ui_worldmap` para la última posición GPS, `ui_chart` para el histórico de la batería.
4. Nodo `switch` sobre `baston/alerta` que dispare una notificación visual.

Instala `node-red-dashboard` y `node-red-contrib-web-worldmap` desde el menú **Manage palette**.

7.4 Criterio de aceptación de la Fase 5

Una caída simulada aparece en el tablero con su ubicación en el mapa en **menos de 5 segundos** (objetivo específico 4). Mide esa latencia con cronómetro en 10 repeticiones y reporta el promedio; es directamente comparable con los 4,71 s de Fitriandini et al. [10].

8. Fase 6 — Alimentación autónoma

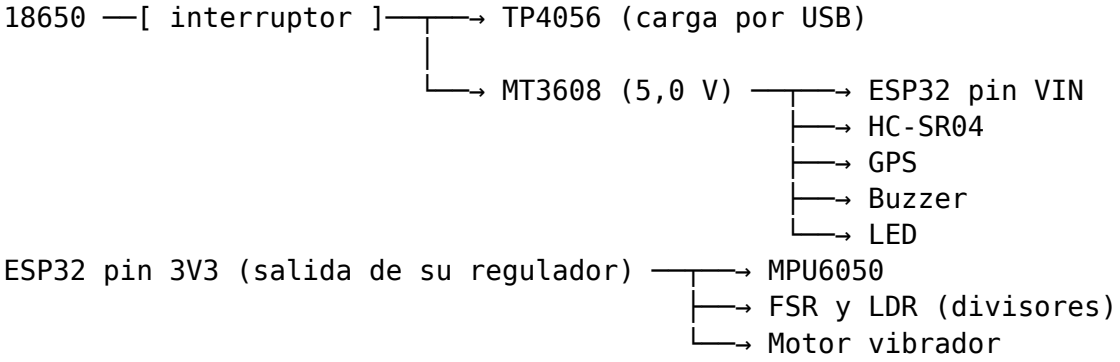
Hasta aquí todo funcionó por USB. Ahora llega la parte donde es fácil quemar algo.

8.1 Ajustar el MT3608 ANTES de conectar nada

El convertidor elevador viene de fábrica con un ajuste arbitrario que puede dar **hasta 28 V**. Si conectas el HC-SR04 sin ajustarlo primero, lo destruyes al instante.

1. Conecta solo la batería a la entrada del MT3608.
2. Mide la salida con el multímetro, **sin ninguna carga conectada**.
3. Gira el potenciómetro multivuelta hasta leer exactamente **5,0 V**.
4. Recién entonces conecta las cargas.

8.2 Cadena de alimentación



No alimentes el pin 3V3 del ESP32 directamente desde la celda. La celda va de 4,2 V a 3,0 V conforme se descarga; por debajo de ≈3,2 V el chip se vuelve inestable y los ADC derivan. La ruta correcta es celda → MT3608 (5 V) → VIN, dejando que el regulador AMS1117 de la placa produzca los 3,3 V.

8.3 Medir la autonomía real

El informe estima 15 horas con 170 mA promedio. Verifícalo:

1. Carga la celda a tope (el LED del TP4056 cambia de rojo a azul).
2. Pon un multímetro en serie en modo amperímetro y anota el consumo en tres condiciones: reposo, marcha normal y durante una alerta.
3. Deja el bastón funcionando en marcha simulada y mide cuánto dura hasta que la protección del TP4056 corte.

Reporta el valor medido junto al estimado. Si difieren mucho, la causa más común es el LED encendido permanentemente o que el modo de bajo consumo nunca se activa porque el umbral del piezo está mal fijado.

8.4 Modo de bajo consumo

Cuando no hay golpes de piezo durante 5 minutos, reduce la frecuencia de muestreo y apaga el GPS. El `light sleep` del ESP32 mantiene la RAM y despierta por interrupción, que es lo que necesitas aquí; el `deep sleep` reinicia el programa y pierde el estado.

9. Fase 7 — Montaje en el bastón

9.1 Distribución física

Zona	Qué va ahí
Empuñadura	ESP32, batería, TP4056, MT3608, PCB principal, FSR, motor vibrador, pulsador, buzzer
Tubo medio, cara frontal	HC-SR04, apuntando al frente con ≈10° de inclinación hacia abajo
Tubo medio	LED y LDR
Interior del tubo, parte alta	GPS, con la antena cerámica hacia arriba y sin metal encima
Contera	Piezo, pegado con epóxico a la cara interna

El GPS necesita cielo despejado sobre la antena. Si lo entierras dentro de una carcasa metálica o bajo la mano del usuario, no fija. Es la restricción de montaje más rígida del proyecto.

El MPU6050 va en la empuñadura, no en la punta: en la punta los golpes normales de la marcha saturan el acelerómetro y arruinan la detección de caídas.

9.2 Orden de armado

1. Imprime las carcasas en PLA y verifica que encajan **antes** de soldar.
2. Pasa todo a perfboard con zócalos de espadines hembra para cada módulo, de modo que puedas reemplazar un sensor sin desoldar.
3. Tiende los cables por dentro del tubo cuando sea posible; lo que vaya por fuera, fíjalo con velcro y bridas.
4. Deja accesible el puerto micro-USB del ESP32 para reprogramar, y el del TP4056 para cargar. Son dos conectores distintos.
5. Prueba el plegado del bastón: el cableado no debe tensarse en las articulaciones.

9.3 Criterio de aceptación de la Fase 7

El bastón se pliega y despliega sin tensar cables, funciona 30 minutos de caminata real sin fallos, y se puede recargar y reprogramar sin abrir la carcasa.

10. Fase 8 — Validación

10.1 Tabla de resultados a llenar

Esta tabla es la que va en la sección de resultados del informe final.

Métrica	Objetivo	Medido	Referencia
Error de distancia (concreto)	< 3 cm		—
Alcance útil de detección	2 m		—
Sensibilidad de caídas	≥ 90 %		—
Especificidad de caídas	—		—
Falsas alarmas por hora de marcha	—		—
Rechazo de caída del bastón solo	—		Aporte propio
Tiempo de primer fix GPS	< 2 min		—
Error de posición GPS	—		4,63 m [10]
Latencia de alerta al tablero	< 5 s		4,71 s [10]
Autonomía	≥ 1 día		≈50 h [9]

10.2 Limitaciones a reportar con honestidad

Mantén en el informe final las limitaciones ya declaradas en los alcances: el sistema mide distancia pero no clasifica el obstáculo; no detecta huecos ni escaleras descendentes; el cono de 15° no cubre obstáculos laterales ni a la altura de la cabeza; el ultrasonido falla con superficies blandas o de vidrio; el GPS se degrada en interiores; es un prototipo académico, no un equipo médico certificado; y las pruebas con un grupo reducido no son estadísticamente generalizables.

Añade una más que surge del cambio de diseño: **el sensor de agarre detecta que hay una mano en la empuñadura, no que esa mano pertenezca a una persona de pie.** Un usuario que cae sin soltar el bastón se detecta; uno que lo suelta en el aire durante la caída, no. Decláralo: es una limitación real y reportarla vale más que ocultarla.

Anexo A — Diagnóstico de fallos

Síntoma	Causa más probable	Solución
No aparece el puerto COM	Cable USB de solo carga	Cambia de cable antes que nada
`Connecting.....____` al subir	La placa no entra en modo bootloader	Mantén BOOT presionado durante la carga
El Monitor Serie muestra basura	Velocidad incorrecta	Ponlo en 115200
Reinicios al activar el motor	Falta el diodo de rueda libre o el capacitor	Agrega 1N4007 y electrolítico en el riel
El ADC se congela al conectar Wi-Fi	La entrada está en un pin ADC2	Muévela a GPIO 32–39 (ADC1)
El MPU6050 no aparece en el escáner I2C	SDA y SCL cruzados	Intercámbialos
La SVM se queda clavada en ≈2 g	El acelerómetro está en rango ±2 g	Configura ACCEL_CONFIG a ±8 g
El GPS no fija nunca	Está bajo techo	Sácalo al aire libre, antena hacia arriba
El GPS tarda 30 s en cada encendido	Batería ML1220 agotada	Normal en módulos clones; documéntalo
El HC-SR04 devuelve 0 siempre	Alimentado a 3,3 V en lugar de 5 V	Conéctalo al riel de 5 V
Lecturas erráticas del FSR	Masa no común o cable muy largo	Unifica GND, acorta el cable, filtra por mediana
El piezo dispara solo	Umbral muy bajo o falta la 1 MΩ	Sube el umbral, suelda la resistencia en el piezo
`ledcAttach` no compila	Estás en el core 2.x	Usa `ledcSetup` + `ledcAttachPin`
El bastón se reinicia al arrancar	Algo conectado a un pin de strapping	Libera GPIO 0, 2, 12 y 15

Anexo B — Cronograma sugerido

Semana	Fase	Entregable
1	Fase 0 + compras	Entorno instalado, componentes pedidos
2	Fase 1 (sensores 1–3)	HC-SR04, MPU6050 y FSR validados
3	Fase 1 (sensores 4–7)	Piezo, GPS, LDR y actuadores validados
4	Fase 2	Integración en protoboard estable 10 min
5	Fase 3	Firmware integrado con máquina de estados
6	Fase 4	Dataset etiquetado y umbrales calibrados
7	Fase 5	MQTT, BLE y tablero funcionando
8	Fase 6 + 7	Alimentación autónoma y montaje en el bastón
9	Fase 8	Pruebas, métricas y tabla de resultados
10	—	Informe final y demostración

Las fases 1 y 4 son las que más se alargan en la práctica. Si vas con retraso, recorta alcance en el tablero (un tablero simple en Node-RED basta), nunca en la calibración: los datos de la fase 4 son el contenido experimental de la entrega.

Anexo C — Lista de verificación final

Antes de la demostración:

- ☐ Batería cargada (LED del TP4056 en azul)
- ☐ Umbral del FSR calibrado para quien vaya a sostener el bastón
- ☐ Umbral del piezo ajustado a la superficie del lugar de la demo
- ☐ Credenciales Wi-Fi del lugar cargadas en el firmware
- ☐ Broker MQTT accesible desde esa red (prueba con MQTT Explorer)
- ☐ GPS con fix confirmado al aire libre antes de empezar
- ☐ Colchoneta disponible si se van a simular caídas
- ☐ Sketches de prueba individuales a mano por si algo falla en vivo
- ☐ Plan B sin red: mostrar la alerta local (buzzer + vibración) si el Wi-Fi falla

Antes de entregar el informe:

- ☐ Tabla II actualizada: FSR reubicado y piezo agregado con sus pasivos
- ☐ Sección III-E ampliada con el principio piezoeléctrico
- ☐ Sección V-D paso 1 reescrita con las dos señales de uso activo
- ☐ Figura 1 regenerada con la entrada adicional
- ☐ Fila “Propuesta” de la Tabla I actualizada
- ☐ Tabla de resultados de §10.1 completa con valores medidos
- ☐ Limitación del sensor de agarre declarada